

Практика

Перед началом выполните создание таблиц из Приложения А в конце файла.

Отправка: В качестве результата предоставить отдельный sql файл для каждого задания с комментариями. Опционально скриншоты.

Задание 1: DML

- **Цель:** Практика базовой вставки, выборки, простого обновления и удаления данных.
- **Действия:**
 1. Вставить двух новых сотрудников в таблицу `Employees` (с любыми отделами, кроме 'IT').
 2. Выбрать всех сотрудников из таблицы `Employees`.
 3. Выбрать только `FirstName` и `LastName` сотрудников из отдела 'IT'.
 4. Обновить `Salary` 'Alice Smith' до 65000.00.
 5. Удалить сотрудника 'Eve Davis'.
 6. Проверить все изменения, используя `SELECT * FROM Employees;`.

Результат: Файл SQL-скрипта, содержащий все DML-операторы, использованные для этих действий.

Задание 2: DDL

- **Цель:** Практика создания и изменения структуры таблиц.
- **Действия:**
 1. Создать новую таблицу с именем `Departments` со столбцами: `DepartmentID (SERIAL PRIMARY KEY)`, `DepartmentName (VARCHAR(50), UNIQUE, NOT NULL)`, `Location (VARCHAR(50))`.
 2. Изменить таблицу `Employees`, добавив новый столбец с именем `Email (VARCHAR(100))`.
 3. Заполнить столбец `Email` для всех текущих сотрудников уникальными значениями (например, через `UPDATE`).
 4. Добавить ограничение `UNIQUE` к столбцу `Email` в таблице `Employees`.
 5. Переименовать столбец `Location` в таблице `Departments` в `OfficeLocation`.

- **Результат:** Файл SQL-скрипта, содержащий все DDL-операторы для этих действий.

Задание 3: DCL

- **Цель:** Научиться создавать пользователей (роли в PostgreSQL) и предоставлять/отзывать базовые разрешения.
- **Действия:**
 1. Создать нового пользователя PostgreSQL (роль) с именем `hr_user` и паролем.
 2. Предоставить `hr_user` право `SELECT` на таблицу `Employees`.
 3. **Тест 1:** В новой сессии подключиться как `hr_user` и попытаться выполнить `SELECT * FROM Employees;`. (Должно сработать).
 - Тест 2:** Под `hr_user` попытаться выполнить `INSERT` нового сотрудника (должна возникнуть ошибка доступа).
 4. Как пользователь-администратор, предоставить `hr_user` права `INSERT` и `UPDATE` на таблицу `Employees`.
 5. **Тест 3:** Как `hr_user`, попробовать выполнить `INSERT` и `UPDATE` сотрудника. (Теперь должно сработать).
- **Результат:** SQL-скрипт, показывающий создание пользователя и операторы `GRANT`. Кратко зафиксируйте успех/неудачу тестовых операций (прикрепить скриншоты).

Задание 4: DML/DCL

- **Цель:** Практика более сложных DML-операций с использованием предложений `WHERE`, множественных обновлений
- **Действия:**
 1. Увеличить `Salary` всех сотрудников в отделе 'HR' на 10%.
 2. Обновить `Department` любого сотрудника с `Salary` выше 70000.00 на 'Senior IT'.
 3. Удалить всех сотрудников, которые не назначены ни на один проект в таблице `EmployeeProjects`. Подсказка: Используйте подзапрос `NOT EXISTS` или `LEFT JOIN`

4. В рамках одной транзакции, вставить новый проект и назначить на него двух существующих сотрудников с определенным количеством `HoursWorked` в `EmployeeProjects`.
- **Результат:** SQL-скрипт, показывающий использование DML и DCL операций.

Задание 5: Функции и представления

- **Цель:** Понять и создать простые SQL-функции и представления в PostgreSQL.
- **Действия:**
 1. **Функция:** Создать функцию PostgreSQL с именем `CalculateAnnualBonus`, которая принимает `employee_id` и `Salary` в качестве входных данных и возвращает рассчитанную сумму бонуса (10 % от Salary) для этого сотрудника. Используйте `PL/pgSQL` для тела функции.
 2. Использовать эту функцию в операторе `SELECT`, чтобы увидеть потенциальный бонус для каждого сотрудника.
 3. **Представление (View):** Создать представление с именем `IT_Department_View`, которое показывает `EmployeeID`, `FirstName`, `LastName` и `Salary` только для сотрудников из отдела `'IT'`.
 4. Выбрать данные из вашего представления `IT_Department_View`.
- **Результат:** SQL-скрипт для создания функции и представления.

Задание 6: DML (Optional)

- **Цель:** Объединение DML-операций с `JOIN`, подзапросами и условной логикой.
- **Действия:**
 1. Найти `ProjectName` всех проектов, в которых `'Bob Johnson'` работал более 150 часов.
 2. Увеличить `Budget` всех проектов на 10%, если к ним назначен хотя бы один сотрудник из отдела `'IT'`.
 3. Для любого проекта, у которого еще нет `EndDate` (`EndDate IS NULL`), установить `EndDate` на один год позже его `StartDate`.

4. Вставить нового сотрудника и немедленно назначить его на проект 'Website Redesign' с 80 отработанными часами, все в рамках одной транзакции. Использовать предложение **RETURNING**, чтобы получить **EmployeeID** вновь вставленного сотрудника.
- **Результат:** SQL-скрипт, содержащий все DML-запросы и операторы **SELECT** для проверки.

Приложение А

SQL

```
CREATE TABLE Employees (  
    EmployeeID SERIAL PRIMARY KEY, -- SERIAL for auto-incrementing integer  
    IDs in PostgreSQL  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    Department VARCHAR(50),  
    Salary DECIMAL(10, 2)  
);  
  
CREATE TABLE Projects (  
    ProjectID SERIAL PRIMARY KEY, -- SERIAL for auto-incrementing integer  
    IDs  
    ProjectName VARCHAR(100) NOT NULL,  
    Budget DECIMAL(12, 2),  
    StartDate DATE,  
    EndDate DATE  
);  
  
CREATE TABLE EmployeeProjects (  
    EmployeeID INT,  
    ProjectID INT,  
    HoursWorked INT,  
    PRIMARY KEY (EmployeeID, ProjectID),  
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID),  
    FOREIGN KEY (ProjectID) REFERENCES Projects(ProjectID)  
);  
  
INSERT INTO Employees (FirstName, LastName, Department, Salary) VALUES  
('Alice', 'Smith', 'HR', 60000.00),  
('Bob', 'Johnson', 'IT', 75000.00),
```

```

('Charlie', 'Brown', 'Finance', 62000.00),
('Diana', 'Prince', 'IT', 80000.00),
('Eve', 'Davis', 'HR', 58000.00);

INSERT INTO Projects (ProjectName, Budget, StartDate, EndDate) VALUES
('Website Redesign', 150000.00, '2023-01-15', '2023-06-30'),
('Mobile App Development', 200000.00, '2023-03-01', '2023-10-31'),
('Internal Tools Upgrade', 80000.00, '2023-05-10', '2023-09-15');

INSERT INTO EmployeeProjects (EmployeeID, ProjectID, HoursWorked) VALUES
(2, 1, 160), -- Bob Johnson (ID 2) on Website Redesign (ID 1)
(4, 1, 120), -- Diana Prince (ID 4) on Website Redesign (ID 1)
(2, 2, 200), -- Bob Johnson (ID 2) on Mobile App Development (ID 2)
(1, 3, 80), -- Alice Smith (ID 1) on Internal Tools Upgrade (ID 3)
(3, 3, 100); -- Charlie Brown (ID 3) on Internal Tools Upgrade (ID 3)

```

Требования к сдаче домашнего задания

Результатом домашнего задания является SQL скрипт, скриншоты.

Для удобства проверки домашних заданий, соблюдайте следующие правила работы с репозиторием Git:

- Каждое домашнее задание необходимо коммитить в отдельную ветку, создаваемые от ветки main.
- Имя ветки должно строго соответствовать формату <lesson_name>_<task_number> (db_normalization_2, python_loops_7)
- Для сдачи задания необходимо создать Pull Request в main-ветку.
- В Pull Request обязательно должен быть добавлен ментор и руководитель лаборатории как reviewers, чтобы они могли проверить и оставить комментарии.
- Не мержите PR, дождитесь проверки от reviewers.